

Sistemes Operatius (SO)

PRÀCTICA 3

Mario VILAR
20392724

David Díez
20392643

22 d'abril de 2023



UNIVERSITAT DE
BARCELONA

ÍNDEX

1	Introducción	2	Exercici 1	2
2	Implementación y pruebas	2	3 Conclusiones	5

1

INTRODUCCIÓN

Esta practica propone unos retos diferentes a los que estamos acostumbrados en esta asignatura. Hasta el momento hemos utilizado Bash Scripting para trabajar a bajo nivel con el Sistema Operativo. Ahora damos un pequeño salto y empezamos a programar con C para trabajar con los procesos y las señales.

En esta práctica se nos pide implementar el tradicional juego del piedra, papel y tijera. En un principio esto puede no tener mucho misterio y ser trivial, por ello se requiere que lo hagamos utilizando procesos y tuberías anónimas. El funcionamiento es el siguiente, vamos a definir un proceso padre y dos hijos. Los hijos son los jugadores y el padre es quien determina quien gana y quien no. El padre se comunica con los hijos mediante señales y los hijos con el padre utilizando tuberías anónimas.

La elección de este tema nos permite centrar nuestros esfuerzos en hacer un buen código y que trabaje tal y como hemos estado viendo en clase.

2

IMPLEMENTACIÓN Y PRUEBAS

Exercici 1. *¿Cómo se ha planteado el código?*

Demostració. Ya partíamos con una base: las funciones `leer_opciones` y `leer_reglas` ya estaban implementadas y teníamos los `struct` de las reglas y las opciones ya creados. De esta forma toda la parte de gestión de ficheros ya la teníamos implementada y no nos teníamos que preocupar.

Por este motivo nosotros nos hemos encargado de implementar todo el resto. En particular hemos trabajado sobre la función `main`, la función `sigusr1`, `jugador` y `arbitro`. Veamos una a una que hace cada una.

1. Empezamos por la función `main`. Como siempre, esta es la función principal del script y se encarga de inicializar el flujo del programa. En esta función recibimos los datos pasados por la línea de comando mediante los parámetros `int argc` y `char *argv[]` y los procesamos para asegurarnos que son correctos y no hay errores.

Una vez nos hemos asegurado que todo es correcto ya estamos preparados para poder inicializar la tubería anónima y establecer quien gestiona las señales. El código para hacer esto es el siguiente:

```
1 signal(SIGUSR1, sigusr1);  
2 pipe(fd);
```

Ahora podemos iniciar los procesos para trabajar con el padre y ambos hijos. El código para realizar esta tarea se compone de un par de `if`. Puede parecer inofensivo pero es muy importante ponerlo bien para evitar que se entre varias veces en la misma condición o que se creen más hijos de los que queremos.

```
1 if((jugador1 = fork()) == 0){
2     jugador(num_opciones, N, fd);
3     return 0;
4 }
5 else{
6     if((jugador2 = fork()) == 0){
7         jugador(num_opciones, N, fd);
8         return 0;
9     }
10    else{
11        arbitro(jugador1, jugador2, num_reglas, N, fd, o_array, r_array);
12        wait(NULL);
13    }
14 }
```

Observamos que hay varios `return` y `wait()`. Esto es precisamente para evitar que pase lo que hemos comentado justo arriba. Una prueba que hemos realizado era poner `printf()` en la entrada de cada condición seguido de `getpid()`, `getppid()`. De esta forma veíamos cuando entrábamos a cada condición y qué hijo era y quien era el padre. Antes de llegar a esta versión final hemos obtenido resultados muy desorbitados. Por ejemplo, la creación de hasta 3 hijos y de 2 padres.

Al acabar estas condiciones no podemos olvidar cerrar la tubería utilizando `close()`.

2. La función `sigusr1` es muy cortita pero super importante para un buen funcionamiento de todo. Esta función tan solo utiliza la variable `varSigusr1` y le cambia el valor de 0 a 1.

Dicha variable se define globalmente y sirve para implementar la espera activa de los jugadores hasta recibir la señal del padre. La importancia de la función `sigusr1` es que cambia el valor de esta variable y así indicar que la espera ha acabado y poder realizar la acción pertinente.

3. La función `jugador` modela qué ha de hacer cada jugador. El funcionamiento es el siguiente: Creamos una semilla para crear un número aleatorio y realizamos un bucle de N iteraciones, donde N es el número de partidas que vamos a jugar. En este bucle lo primero que hacemos es esperar a recibir la señal del padre conforme podemos elegir que jugar. Una vez decidido se lo enviamos por una tubería utilizando `write()`.

En esta función las pruebas más relevantes fueron comprobar que entramos con el hijo correcto en el momento correcto ya que en las primeras versiones del código los hijos iban dando saltos. Por ejemplo, primero decidía el `jugador2`, después el `jugador1` y volvía el `jugador1`.

4. Por último, la función `arbitro` se encarga de gestionarlo todo. Obsevamos que además es una función que recibe 7 parámetros. Esto es, me repito, porque es el principal gestor del juego.

A efectos prácticos lo que hacemos en esta función es iniciar un bucle para representar las N partidas. En cada una de sus iteraciones, que simula un único juego, lo que hacemos es utilizar la señal `sigusr1` y la función `kill()` para dar permiso a cada uno de los hijos a dar su elección. Con tal de guardar la elección de cada uno utilizamos `write()` y la variable `fd` que hemos definido en el `main()` y que obviamente hemos pasado por parámetro mediante punteros.

Una vez tenemos las elecciones de cada uno de los jugadores lo que hacemos es utilizar la matriz `o_array` para obtener que objeto utiliza. Lo tenemos codificado de la siguiente manera: El 0 representa a la piedra, el 1 al papel y el 2 a la tijera. Para obtener esta información tenemos que utilizar la línea

```
1 strncpy(movimiento, o_array[eleccion1].nom, MAX_LEN - 1);
```

Si analizamos esta línea vemos que estamos utilizando la función `strncpy` para guardar en la variable `movimiento` el nombre del elemento en la posición `eleccion1` de la matriz de opciones.

Una vez tenemos todos los datos guardados ya podemos procesarlos y determinar quien es el ganador. Para ello tan solo hay que hacer varios `if`. El primero de todo es el siguiente.

```
1 if(eleccion1 == eleccion2){
2     printf("%s anula %s\n", movimiento1, movimiento2);
3     printf("Han empatado!\n");
4     empate++;
5 }
```

Utilizando esto comprobamos si ambos jugadores han jugado lo mismo y por lo tanto han empatado. Merece la pena fijarse en las tres líneas dentro de la condición.

La primera imprime por pantalla como ha sido el desarrollo de la partida, la segunda el resultado obtenido, que en este caso ha sido empate, y la tercera suma uno a un contador. Este tipo de contador lo veremos más adelante y sirve para dar un resultado general de las N partidas.

Una vez nos hemos asegurado que los jugadores no hayan empatado podemos pasar al siguiente código:

```
1 for(int j = 0; j < num_reglas; j++){
2     if(eleccion1 == r_array[j].id1 && eleccion2 == r_array[j].id2){
3         strncpy(descripcion, r_array[j].descripcion, MAX_LEN - 1);
4         printf("%s %s %s\n", movimiento1, descripcion, movimiento2);
5         printf("Ha ganado el Jugador 1\n");
6         victorias1++;
7     }
8     if(eleccion2 == r_array[j].id1 && eleccion1 == r_array[j].id2){
9         strncpy(descripcion, r_array[j].descripcion, MAX_LEN - 1);
10        printf("%s %s %s\n", movimiento2, descripcion, movimiento1);
11        printf("Ha ganado el Jugador 2\n");
12        victorias2++;
13    }
14 }
```

En este caso estamos haciendo un bucle para recorrer todas las reglas que tenemos. En cada iteración comprobamos si da la casualidad de que el jugador1 haya elegido el primer elemento y el jugador2 el segundo ya que en tal caso el jugador1 habrá ganado al jugador2 o viceversa.

Podemos volver a ver que dentro de cada condición utilizamos las mismas líneas que antes, es decir, la función `strncpy` para guardar en `descripcion` el texto pertinente, imprimimos por pantalla el desarrollo de la jugada seguido de la resolución y sumamos 1 al contador que toque.

Por último, ya fuera del bucle, es decir, al haber acabado las N partidas. Mostramos el resultado general obtenido por ambos jugadores. Para hacer esto utilizamos los contadores que hemos ido aumentando en las condiciones anteriores. El código es tan simple como

```
1 printf("\nLOS RESULTADOS DE LAS %d PARTIDAS HAN SIDO:\n", N);
2 printf("El Jugador 1 ha ganado %d partidas\n", victorias1);
3 printf("El Jugador 2 ha ganado %d partidas\n", victorias2);
4 printf("Han empatado %d veces\n", empate);
```

Esta función es más extensa y complicada que el resto, por ello también hemos tenido más problemas que en las otras. Uno de los más grandes que tuvimos fue haber implementado espera activa aquí también. Esto hacía que el hijo esperara al padre y el padre esperara al hijo, de esta forma el programa quedaba parado esperando a una señal que nunca llega. La solución era obvia, quitar la espera activa del padre. Otro problema, que fue una tontería y nos hizo perder mucho tiempo fue un `exit(0)` al final de la función. Esto hacía que no siguiera el flujo correcto y quedara parado de forma inesperada. Por último, quería dejar apuntadas las líneas necesarias para ejecutar el código en C.

```
1 gcc -c Practica3.c
2 gcc Practica3.o -o Practica3.exe
3 ./Practica3.exe opciones.csv reglas.csv 3
```

Por último queríamos hacer un pequeño comentario. En el `.zip` hemos adjuntado dos archivos de reglas y dos de opciones. Los archivos básicos corresponden al juego piedra, papel y tijera tradicional mientras que los archivos con el sufijo *Extra* hacen referencia al piedra, papel y tijera versión *Big Bang Theory*. ✓

CONCLUSIONES

En conclusión, esta práctica nos ha servido para tener un primer contacto con la comunicación entre procesos y las señales. Así como poder ver de una forma más práctica como interactúan varios hijos con un mismo padre y como se trabaja con tuberías anónimas. Hemos aprendido mucho y hemos visto la importancia que tiene en el mundo de la informática, especialmente porque nos permite trabajar entre varios procesos. En definitiva, tenemos ganas de seguir aprendiendo.